

FIT3143 Lab #2 Assessment (Week 8)

MESSAGE PASSING INTERFACE

OBJECTIVES

- The purpose of this lab assessment is to introduce you to Open MPI
- Design and analyse distributed parallel algorithms with the Message Passing Interface (MPI)

MARKS

- The lab session is worth **8 marks** of the unit's final mark.

LAB INSTRUCTIONS

1. Preparation is required for this Lab session. Please **DO NOT** plan to complete the Lab without any preparation/understanding.
2. Students should start working on the Lab tasks at least 1 week prior to the assessed session.
3. Students must submit their answers and codes via Moodle in the required format before the start of the allocated class. A **late penalty** (5% per day) will apply if you do not submit the required materials on time!
4. In addition to regular offline/off-class marking, students' (or Team's) slides, answers, and/or source code will also be reviewed and assessed through in-class presentations, peer reviews, and/or question-and-answer sessions.
5. For **team/group presentations**, marks in general will be allocated for:
 - The correctness of the codes or results
 - The quality and clarity of the presentation
 - The correctness of the explanation during the presentation

The presentation content should include slides that are clear, easy to understand, and readable in a professional working environment (aka an industry setting). While the teaching team does not impose strict limits on the presentation format (slides/document format), nevertheless, if the presentation is hard to read/digest (e.g., speech is misaligned with the presented content, or slides/speech are unclear, or arguments are

made without sound logic), marks will be deducted. Both the quality and correctness of your submitted/demonstrated work are assessed and graded.

6. For **individual/group interviews and Q&A sessions**, marks will be allocated for:

- The correctness of the verbal answer
- The quality of the verbal answer

Focus on the most important idea in the answer. Marks will be awarded on the student's ability to explain their ideas with reference to the submitted code or documents in a concise, focused and accurate manner. Answers with errors, inaccurate or lengthy explanations will lose marks. Showing little to no understanding of the code will result in a loss of marks. Showing little to no understanding of the associated libraries, or submitting results will result in a loss of marks. Showing little to no understanding of how to select and filter information (aka dumping unfiltered answers without processing from the textbook/internet/AI agent) will lose marks.

- 7. Always make sure you read and follow the marking rubric well in advance of the deadline.
- 8. Marks will not be awarded if you skip the class, or do not make any submissions, or do not make any contributions in the working group/team, or make an empty submission to Moodle (unless a special consideration extension has been approved).
- 9. You are allowed to use Generative-AI to search for information and resources during the preparation period. However, you must declare in your report and upload all the prompt records (in PDF files).
- 10. AI tools are not allowed during the presentation period or any oral/coding interview sessions.
- 11. All submitted files should ideally include students' names, ID and Monash email addresses.

LAB IN-CLASS ASSESSMENT ACTIVITIES

Students are required to form teams of 2.

Note: For students who cannot form teams due to exceptional circumstances, arrangements must be made with the teaching team (generally the lead staff member in the corresponding applied/lab class) on/before Week 7.

- ❖ Team Preparation Period (before deadline): Divide the workload among team members and complete all the tasks. For each team, you must submit the required files (e.g., report, source code, slides, and corresponding documents) before the deadline.
- ❖ Team Presentation Period (max 7 min): For each team, you are required to present and demonstrate your code, results and observations during the lab session. All team members must be present in the lab session.
- ❖ Q&A Period (approx. 2 min): Each team member will be asked roughly two questions based on the submitted and/or presented work.

Note: If the required submissions are missing or any member(s) of the student team are absent, no marks will be given to the assessment.

Task 0 - Getting used to Open MPI manual pages

Implementing Open MPI programs generally involves utilising MPI functions involving complex function arguments. Make sure you are familiar with using the manual pages <https://docs.open-mpi.org/en/v5.0.x/man-openmpi/index.html>. Make sure it sits comfortably in your browser, and you can quickly access the API descriptions whenever you need them. You should have completed the activities in Week 7's lab before proceeding to complete the following tasks in this lab specifications. Additionally, please refer to the CAAS documentation or setting up your own local cluster prior to completing the following tasks. This is important for performance analysis of your parallel code beyond a single computer.

Task 1 – Prime Search using Message Passing Interface (Open MPI)

Let us revisit the prime search problem, which you previously worked on in Week 4 (Lab #1).

In Week 4:

“You were required to write a *serial* C program to search for prime numbers that are strictly less than an integer n , provided by the user. The program will output a sorted list of all prime numbers found.

Example: For instance, if the user inputs n as 10 on the terminal, the prime numbers being printed are: 2, 3, 5, 7 (sorted in ascending order).”

“In week 4, you were also required to implement a parallel version of your serial code in C utilising *POSIX Threads* (Task 2) and *OpenMP* (Task 3), by designing parallel partitioning schemes to distribute the workload and implement in C.”

In Week 8, we will continue with the same old prime searching problem from Week 4. You are required to implement a parallel version of your serial code in C using the **Message Passing Interface with Open MPI**.

Coding requirements:

- The root process reads the integer n value (e.g., $n = 10,000,000$ or higher depending on your hardware) as a command-line argument.
- After obtaining the value n from the user, the value is required to be disseminated to all the other MPI processes.
- Each process (including the root process) will be tasked with a share of the workload to compute the prime number. You will need to design your own workload distribution and explore/experiment with different workload distribution options to find the best approach. You should design and implement an efficient and balanced workload distribution algorithm.
- After the result is computed by all the processes, the root process will print the final result to a text file. The prime numbers must be in correct sorting order.

Note: You are advised to modify the code to ensure it no longer asks the user for the value of n at runtime. Instead, the value of n should now be passed as a command-line argument to the application. Alternatively, the root process can read the value of n from a text file and broadcast this value to all other processes within the MPI communicator for the workload distribution and parallel prime search. You should run your program on a single computer and across multiple computers using CAAS or your own locally setup cluster.

Reminder: *Your Open MPI code must be clean and bug-free so you can reliably compare your Open MPI version (Task 1) against the serial version (Week 4 Task 1), the POSIX Thread version (Week 4 Task 2), and the Open MP version (Week 4 Task 3). The code should also be well-formatted so that you can easily explain your approach and/or demonstrate your code.*

Reminder: Make sure you execute and test your compiled program with different numbers of MPI processes and different workload distributions using your machine with Docker, a Linux virtual machine, or a native Linux machine.

Common questions you need to consider:

- What is the speed-up? Is it reasonable? If not, is the run time measurement correct?
- How would the speed-up change when you increase & decrease the number of MPI processes and the size of n ? Why?
- How do you distribute the tasks to ensure balanced workload distribution? Is it a good approach?

Task 2 – Prime Search using hybrid OpenMP and Message Passing Interface (Open MPI)

In this task, you are required to implement a hybrid version combining shared memory parallelism with OpenMP and distributed memory parallelism with OpenMPI.

Coding requirements:

- The root process reads the integer n value (e.g., $n = 10,000,000$ or higher, depending on your hardware) as a command-line argument.
- After obtaining the value n from the user, the value is required to be disseminated to all the other MPI processes. The value should be accessible by all the threads within the same process.
- Each MPI process (including the root process) will create a set of threads to compute the prime numbers in parallel. You will need to design your own workload distribution and explore/experiment with different workload distribution options to find the best approach. You should design and implement an efficient and balanced workload distribution algorithm using a different number of threads and processes.
- After the result is computed by all the threads from all the processes, the main thread of the root process will print the final result to a text file. The prime numbers must be in

correct sorting order.

Note: You are recommended to modify the code to make sure the code no longer asks the user for the value of n at runtime. Instead, the value of n should now be passed as a command-line argument to the application. You should also run your program on a single computer and across multiple computers using CAAS or your own locally setup cluster.

Reminder: *Your code must be clean and bug-free so you can reliably compare your hybrid version (Week 8 Task 2) against the serial version (Week 4 Task 1), the POSIX Thread version (Week 4 Task 2), the Open MP version (Week 4 Task 3), and the Open MPI version (Week 8 Task 1). The code should also be well-formatted so that you can easily explain your approach and/or demonstrate your code.*

Reminder: Make sure you execute and test your compiled program with different numbers of threads, different number of MPI processes, and different workload distributions using your machine with Docker, a Linux virtual machine, or a native Linux machine.

Common questions you need to consider:

- What is the speed-up? Is it reasonable? If not, is the run time measurement correct?
- How would the speed-up change when you increase & decrease the number of threads, MPI processes, and the size of n ? Why?
- How do you distribute the tasks to ensure balanced workload distribution? Is it a good approach?

Task 3 – Performance evaluation with Amdahl's Law

In this task, you will need to perform empirical evaluations and theoretical analysis (Amdahl's Law) for your implemented task 1 and task 2.

For empirical evaluations, you are required to measure the overall wall-clock time and **derive the empirical speed-up** for both task 1 and task 2 against the serial code (Week 4 Task 1) with:

- Increasing problem size n ,
- Increase the number of MPI processes, and
- [Task 2] Increasing number of threads (per MPI process).

Note: You should always compare your results and derive the speed-up against the serial version (implemented in Week 4 Task 1). Make sure you also consider and include the information on the number of processors (cores) available in your machine.

For theoretical analysis, you are required to design experiments to measure the wall-clock time of both the serial part(s) and parallel part(s) for your implemented code in Task 1 and Task 2, and **utilise Amdahl's Law or the Gustafson's Law to compute the theoretical speed-up** with:

- Increasing problem size n ,
- Increase number of MPI processes, and
- [Task 2] Increasing number of threads (per MPI process).

Note: You will need to compute the serial fraction and the parallel fraction for Amdahl's Law or the Gustafson's Law. Note that these two laws have different measurement assumptions, and you will need to design measurement experiments to compute the serial/parallel fractions

correctly.

Some questions to consider:

- How does the actual speed up compare against the theoretical speed up?
- Will more MPI processes always increase the actual and/or theoretical speedup?
- How would the workload distribution affect the speed up?
- Will the speed-up results be the same across different (team members') machines?

Task 4 – Presentation slides and documentations

In this task, you are required to prepare presentation slides (or documentation) to report and present your answers and results from previous Tasks. You will have 7 minutes to present your slides and about 2 minutes for the Q&A session.

Your presentation slides (or documentation) should have the following sections and content (at minimum):

a) (Task 1) Open MPI code [Estimated Presentation Time: 3 min]

The section introduces your approach to implementing the distributed version with Open MPI. You are required to discuss your parallel partitioning scheme (workload distribution) and produce graphs comparing your Task 1 implementation against your serial implementation in Week 4 Task 1 and your parallel implementation in Week 4 Task 2 & Week 4 Task 3.

You are also required to produce the following graphs:

- 1) Comparing the run time of your Open MPI implementation against your POSIX Threads or OpenMP implementations, with increasing size of n .
- 2) Showcasing the empirical speed up of your Open MPI implementation against your POSIX Threads or OpenMP implementations, with increasing size of n .
- 3) Comparing the empirical speed up of your Open MPI implementation against your POSIX Threads or OpenMP implementations, with an increasing number of Open MPI processes (and the same number of threads for POSIX Thread/OpenMP implementations).

b) (Task 2) Open MPI + Open MP code [Estimated Presentation Time: 2 min]

The section introduces your approach to implementing the hybrid version. You are required to discuss your parallel partitioning scheme (workload distribution) and produce graphs comparing your Task 2 implementation against your Task 1 implementation.

You are also required to produce the following graphs:

- 4) Comparing the empirical speed up of your hybrid implementation against your Open MPI implementations in Task 1, with an increasing number of threads (and the same number of Open MPI processes for your Task 1 implementations).
- 5) Comparing the empirical speed up of your hybrid implementation against your POSIX

Threads or OpenMP implementations, with an increasing number of Open MPI processes and threads. Your POSIX thread/OpenMP comparison should have the number of threads matching with the total number of threads created for your hybrid implementation. As an example, you need to create 6 threads for your POSIX Thread / Open MP implementations if you want to compare the hybrid implementation with 3 Open MPI processes, each with 2 threads.

c) (Task 3) Performance evaluation [Estimated Presentation Time: 2 min]

The section presents your theoretical analysis on your Task 1 and Task 2 implementations with Open MPI. You are required to discuss how you design experiments to measure the wall-clock time of both the serial part(s) and parallel part(s) for your implemented code in Task 1 and Task 2. You are also required to show how you use the measurements to derive the right parameters for the Amdahl's / Gustafson's Law.

You are required to produce the following graphs:

- 6) Comparing the empirical speed up and theoretical speedup on your Open MPI (Task 1) implementations with an increasing number of Open MPI processes.
- 7) Comparing the empirical speed up and theoretical speedup on your hybrid (Task 2) implementations with an increasing number of Open MPI processes and an increasing number of threads.

Note: To obtain statistically convincing evidence, you should test with at least 30 different numbers of n . You may also need to increase n further to obtain significant results (depending on your machine hardware). It is best to avoid showcasing runtime results that are too small (e.g., < 1 seconds) that can be easily be affected by measurement noises (e.g., influences by other background processes).

Note: Make sure you test the number of threads and Open MPI processes from 1 up to at least the number of cores available in your CPU. You should also try to see if you can set up MPI processes across team members' machines. What would happen if you increase the number of Open MPI processes / threads more than the number of cores available in your machine?

d) Q&A [Estimated 1 to 2 min]

The section invites questions from the audience (TA/markers). The teaching team will ask questions related to your presentation and the submitted files.

Note: Practice makes perfect. If you are still deciding what to present and searching for information/content from the slides, chances are you will not be able to present within the time limit.

Note: You are allowed to put extra slides as an appendix (after the Q&A section) for your own convenience. Make sure they are clearly labelled so that markers will not be confused or mix them up with the other required sections.

Submission Checklist

- Coding Tasks (Task 1 and 2) – task1.c and task2.c
- Task 4 - Presentation slides (or documentation)
 - The documentation can be in the format of slides, docx, PDF or an equivalent format.
 - Please aim to keep the slides or documentation to within a 7-minute presentation (excluding the Q&A session).
 - Focus on emphasising the design, results and observations that would demonstrate a good understanding of the lab tasks.
- Task 3 (Optional) – Any additional notes/documents to support your calculations and experimental design.
- Separate AI declaration files (in PDF format), **if not** already declared in the submitted documentation as aforementioned.

LAB INSTRUCTIONS FOR ALTERNATIVE ASSESSMENT

This section only applies if you or your team member(s) are **granted a special consideration extension** by the special consideration team (i.e., with a valid approval notice).

1. As a reminder, note that the regular 2 day extensions would not apply for applied/lab sessions (as they are in-class assessments with group/team works).
2. Instead of reviewing student's answers (in-class) in a presentation, peer review, and/or question and answer format during the regular in-class session, **student's answers (Task 4) will be assessed based on a recorded presentation video**. The video is required to be less than 9 minutes, and markers will be marking the assessment offline.
3. Markers will only mark the first 9 minutes of the video at regular playback speed. If students decide to compress the video by speeding up the playback speed, presentation quality could be affected and marks may be deducted according to the marking rubric.
4. Instead of having an in-person Q&A sessions, students will need to answer the following two questions in their recording:
 - Would you recommend Open MPI for prime number searching (against POSIX Thread/Open MP)? Why/Why not? What are the pros and cons of Open MPI on prime number searching (against POSIX Thread/Open MP)?
 - Is there a difference between the empirical speed-up and the theoretical speed-up? Why/Why not? Explain your reasons.
5. Apart from the recorded video file (mp4), students are still required to submit the presentation slides (PDF/pptx), source codes, and any AI declarations/reports (PDF) listed in the regular submission checklist via Moodle before **the approved extended due date**.
6. **A late penalty** (5% per day) will still apply if you do not submit the required materials on time (against the new extended date)! As a reminder, if you submit 7 days after the extended due date, you will still receive a 0 mark.
7. AI tools are generally not allowed during the recording sessions.
8. Before you start your presentation, you are required to show your face and your student ID in the video.